



The Open University

M263 Building Blocks of Software

Unit 16

Looking forward

Study guide

This unit has a quite different role and style from earlier units in the course. It is intended as a pointer to some possible topics for future study. The style is mostly discursive. The only section with technical detail to study is Section 2, and it is not expected that you will spend time trying to gain a thorough understanding of the material. The other sections contain mainly read-only material. Treat each of Sections 1–4 as a separate item. They are brief contributions by different authors — do not expect them to read as a single coherent unit.

The final section gives a number of references. These are there for your information and interest — pursuing these references is not intended to be part of your study in this course!

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwenq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2005. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Huddersfield.

ISBN 978 07492 2306 9

Contents

Introduction	4
1 Specification and the programming industry	5
2 Algebraic specification of a type	7
3 Fuzzy logic	18
4 Beyond big Oh	23
5 References and Further Reading	25
Solutions to the exercises	27
Index	28

Introduction

The earlier units of this course have introduced a wide range of topics. Many of these have been dealt with rather briefly. In this unit, we will indicate ways in which some of the topics might be taken further. The coverage in this unit is in no way systematic — it is a brief introduction to possible lines of development for certain topics.

We have emphasised the importance of specification. The specifications we have given have usually expressed the precondition and postcondition in natural language (i.e. informally). We have indicated how one might try to express specifications formally, in Units 10 and 13. We shall go further in this unit, and outline how we may specify a new (class) type. To do this, we will express the behaviour (semantics) of the methods of the class using what are called **axioms**. These show the behaviour by stating how the methods must interact with each other.

The formal expression of specifications raises subtle problems, which the approach used in this course cannot deal with fully. For example, there is a problem with ensuring that the formal propositions one uses in postconditions always use methods in a way that is consistent with their preconditions. (In Unit 10, we avoided discussion of this.) There are a number of specification languages that are used in practice, such as Z (read as “Zed”) and VDM (short for “Vienna Development Method”). We shall not describe them here, but their role is mentioned.

In Block 3 we introduced formal logic. This is a topic that can be developed in a number of ways. There is a class of programming languages, of which Prolog is an example, which enable one to assert that certain propositions are **true** (in a particular context), and also to give general rules for generating further **true** propositions. (These rules are based on formalisation of perceptions such as this: if x is an uncle of y then x is male, and involve the use of predicates.) In doing this, there will be propositions that are known to be **true**, propositions that are known to be **false**, and propositions for which no truth value is known. The logic we described in Block 3 is **two-valued** — any proposition has a truth value that is either **true** or **false**. One line of development is to a **three-valued** logic (with possible truth values **true**, **false** and **undefined**, for example). In Section 3 here, we look briefly at another development, namely to **fuzzy** logic.

The ideas concerning efficiency introduced in Unit 15 can be taken much further, and we give a brief introduction to the topic in Section 4.

There are many other important areas, so we cannot be in any way comprehensive here. For example, **concurrency** covers processes that occur simultaneously, with the possibility of communication between such “parallel” processes. This is an important area, and requires approaches different from any discussed in this course. Nor have we looked at fundamental issues in the theory of computation, of which a flavour can be obtained from **Church-Turing**.

These may be referred to as **declarative** languages.

In the text, references will be given in this style.
See Section 5 for full references.

1 Specification and the programming industry

In the 1970s it was widely acknowledged that the programming industry was facing a “software crisis”, characterised by software projects overrunning production time, failing to meet requirements and being plagued by bugs. This led to wide adoption of software engineering techniques, encompassing **requirements gathering** (where programmers talk to clients and establish what the software ought to do), **specification** (where the behaviour required of software is stated more carefully), **implementation** and **testing** stages. Within the area of program specification, some practitioners and researchers were investigating methods based on mathematical descriptions of how programs ought to work. This **formal specification** held out the promise of being able to describe a program so completely that we could then prove that it would behave correctly. Perhaps one then could also generate a correct program implementation from a specification.

Practitioners of formal specification campaigned for wider adoption of these mathematical techniques. In areas such as safety critical applications (for example industrial processes or fly-by-wire systems) the need for a principled verification of software behaviour was clear. For example, software failure during radiotherapy could lead to the death of a patient; a plane whose rudders failed to work would be dangerous.

Even so, formal specification was largely ignored in the wider IT industry due to its complexity, the amount of time it takes to do, a lack of user-friendly tools, and the difficulty in scaling up the methods to large problems. We have also mentioned in Unit 14 how proof is a social process. Most of the time, an informal understanding of how a program works has been seen as sufficient “proof” of correctness. A program, after all, is a form of specification in itself, for the behaviour of a computer under its control.

The picture somewhat changed in 1994, when Thomas Nicely reported a bug in the floating-point arithmetic calculating unit (FPU) of the Intel Pentium processor that would cause certain division operations to produce incorrect results. Although the bug was sufficiently rare in practice that it would not affect most people, the public naturally questioned whether they could trust the results of their spreadsheet calculations. Besides, what else might be wrong with their software, waiting to cause a disaster, if something as apparently simple as division of numbers could be wrong?

Nicely is a professor of mathematics at Lynchburg College in the USA.

Here is an excerpt from his original announcement (**Nicely**).

It appears that there is a bug in the floating point unit (numeric coprocessor) of many, and perhaps all, Pentium processors.

In short, the Pentium FPU is returning erroneous values for certain division operations. For example, $1/824633702441.0$

is calculated incorrectly (all digits beyond the eighth significant digit are in error). This can be verified in compiled code, an ordinary spreadsheet such as Quattro Pro or Excel, or even the Windows calculator (use the scientific mode), by computing

$(824633702441.0) * (1/824633702441.0),$

which should equal 1 exactly (within some extremely small rounding error; in general, coprocessor results should contain 19 significant decimal digits). However, the Pentiums tested return

0.999999996274709702

for this calculation.

One could remove the bug by disabling the maths coprocessor, but this would result in a much slower operation overall of any maths-intensive programs. Eventually, Intel was forced to offer replacement processors to its customers, at a very large cost.

This was a wake-up call to some parts of the IT industry that methods which could prove correctness of programs were going to be necessary. A survey paper (**Gerhart**) cites over sixty industrial projects making use of formal methods. Often the techniques are applied to particular critical parts of project code, so avoiding issues with scaling the technique to large projects.

In Unit 10 we mentioned that formal specification was used to prove the correct functioning of a Pentium processor. This technique was performed on a large scale by Intel for the first time with the Pentium 4, which appeared in 2000. Up to this point, most testing effort had been devoted to simulating the behaviour of a processor, but as the number of logic gates rose to the level of hundreds of thousands, it became unsafe to rely on simulation alone as a testing procedure.

Specification, derivation and verification

In this course you have been presented mainly with what we call **implicit specification**. A specification states how a program ought to behave in terms of its inputs and outputs. We have given preconditions and postconditions that, for functions in our course code, describe the relationship between the input and output values. We call this implicit because it is separate from the implementation of the function in code. It only states correct behaviour of a function in terms of inputs and outputs. In common with other formal specification methods, much ambiguity due to the looseness of everyday language (in our case, English) can be removed by translation to mathematical forms.

In **explicit specification**, we observe code at the implementation level, and determine whether it performs correctly according to given preconditions and postconditions. One of the earliest and most enduring examples of this is given by C. A. R. Hoare and R. W. Floyd (see **Hoare2** for example). This approach is also known as an **axiomatic specification** technique. By “axiomatic” we mean that it is possible to write down rules which can be used in proving the correctness of the behaviour of a program. Proving correctness of programs is known as **verification**.

The rules used in axiomatic specification are dependent on the implementation language, but most languages exhibit certain concepts in common with one another, for example the notions of assignment of a value to a variable, a branching (if-then) structure and a repetition (for-loop) structure. Other axioms we can state relate to the way the programming language implements arithmetic operations such as addition or multiplication, taking into account that arithmetic in computers does not allow for infinite precision.

Hoare described a specification structure now known as a **Hoare triple**, which consists of three parts: a precondition (P), a program (Q) and a postcondition (R). The notation $P \{Q\} R$ says that if the condition P is true before the program Q is run, then the condition R will be true when the program has finished. An example is the assignment axiom, which is an axiom about how a statement involving assignment will behave. This axiom then allows one to infer the correct state of a program before an assignment in order to meet a postcondition after the assignment. For example, the triple $P\{x \leftarrow x + 1\}(x == 3)$ implies that, before x is incremented by the

assignment statement, it must have a value 1 less than 3 in order for the postcondition to be met. From this we can deduce that this part of the program is correct if $P \Rightarrow (x == 2)$ is **true**. Likewise, axioms can be stated for other programming constructs. It is possible, therefore, to prove that programs meet a specification.

It is also possible to start with a specification of how a program should behave and thereby determine the code that will meet the specification. This is known as **program derivation**.

Much of the early work in formal methods was devoted to verification, that is, proof of correctness of programs, but currently most interest centres around specification. Two of the most popular specification languages are Z (see **Jacky**) and VDM (see **Jones**). These are not programming languages: they cannot be executed in the way code can. Instead, they describe how parts of programs work, and the states (in terms of values of variables used in the code) that the program ought to be in before and after code is executed. This is separate from a choice of programming language and paradigm (for example, object orientation).

Other specification languages include object-oriented constructs or concurrency (see **Hoare1**, **Milner**). Hoare's CSP (Communicating Sequential Processes) was an early concurrency specification language, while the pi-calculus and the calculus of communicating systems (CCS) are two more recent developments in this area. A process modelling language currently enjoying attention is BPEL4WS, the Business Process Execution Language for Web Services, which is from Microsoft and IBM. This allows specification of business processes and interactions.

See **BPEL4WS**.

2 Algebraic specification of a type

In Blocks 1 and 2 there was an emphasis on the separation of specification and implementation, both for user defined functions and for types. The specifications of the primitive types and of classes were given in terms of their type functions or methods. The specifications we gave are not formal — the form of data of each type, and the specifications of the type functions or methods, were expressed in natural language, not in any formal way. The concept of an abstract data type was mentioned in Unit 4 (p. 23), but we have not considered the formal specification of a user defined class. In this section we will explore the concept further. The following is from **ADT**.

We have given full implementations of some classes.

Abstract data types or ADTs are data types described in terms of the operations they support — their interface — rather than how they are implemented.

Meyer (p. 53) gives criteria for a description of an abstract data type: it should be *complete*, *precise* and *unambiguous*, but *not over-specified*.

So we want a description that:

- describes the data type precisely;
- is as complete as possible;
- achieves the first two criteria without over-specifying.

In particular, the description should avoid reference to any particular form of implementation (either in terms of the way the form of data is represented, or the way in which the type functions or class methods are implemented).

There are a number of approaches to such a formal specification. These approaches use what are called formal methods, which employ specifically designed formal languages based on mathematics, such as Z or VDM. These formal methods are not only used to produce precise specifications but also for formal verifications.

In this section we will look at an approach to the formal specification of an abstract data type, often referred to as **algebraic specification**. Rather than introducing a specifically designed specification language, we shall just use the notation of formal logic introduced in Block 3. (This approach has some limitations. In particular, there is a problem dealing with non-trivial preconditions. We shall avoid such problems by only using methods with precondition **true**.)

We illustrate this approach by specifying a particular abstract data type, **StackC of Int**. This is a class type. It has similar processes to the primitive type **Stack of Int** described in Unit 3, but, being a class, has methods rather than type functions.

Before giving a formal specification we first discuss this class in an informal way. This discussion certainly does not satisfy the criteria just discussed, but it motivates and provides a concrete feeling for the abstract description that we shall develop.

A very loose description of an object taken from the class **StackC of Int** is as follows.

A stack object is a collection of data (in this case integers) kept in a sequence. Each element of data in the collection is from the same set (in this case *Int*). Data is added to and removed from the sequence at one specific end of the sequence (usually called the top). It is possible to access only the top element in a stack object.

The above description is not very precise, but it does give the essential flavour of what a stack object is.

A better specification is one that specifies each of the methods of the class. Specifications of the type functions *PEEK*, *PUSH*, *POP* and *SIZE* of the primitive type **Stack** were given in Unit 3. Here, we shall be concerned with methods of a class, but the behaviour will be similar.

Again, the specifications we shall write down at this stage will not be very precise, but they should be readily understood from our previous work on **Stack** in Unit 3.

To represent a stack we shall use the notation $[n_1, n_2, \dots, n_{k-1}, n_k]$ to indicate a stack of k integers, where n_k is at the top of the stack, and with $[]$ to indicate the empty stack. This notation should be considered just as an aid to help us to understand the behaviour of the methods rather than as any indication on how the data should be stored.

The methods are then roughly specified as follows. All the methods of **StackC of Int** will have precondition **true**. (The postconditions are implicitly considered but no attempt is made yet to make them formal.) The first method, *new*, has no parallel in the type functions given for the primitive type **Stack**. It relates to the creation of a new stack, which is done in the course code through a **var** declaration.

Some authors would insist that a class includes an implementation. Here we are concerned only with specifying the behaviour of the class — and quite definitely want to avoid giving a particular implementation.

Rough specification

1. *new()* is a method that returns an empty stack, that is, a stack with no elements. The method needs no receiver and has no input parameters.

The statement in course code: **var s in StackC of Int** should be thought of as follows. It invokes a method that requires no receiver and that returns a stack object with identifier *s*, which is in the default state of the class (i.e. an empty stack []). Such a method is often referred to as the **constructor** for the class.

2. *push(elem)* is a method that, given an element (*elem*) and a stack object (say in the state $[n_1, n_2, \dots, n_{k-1}, n_k]$), returns a new stack object obtained from the receiver by inserting *elem* into the top position. (So the value returned by *push(elem)* will be $[n_1, n_2, \dots, n_{k-1}, n_k, elem]$.)
3. *pop()* is a method that, for a non-empty stack (say in the state $[n_1, n_2, \dots, n_{k-1}, n_k]$), returns a new stack obtained from the receiver by removing the top item. We need also to consider what should happen if the receiver stack is empty. In this design we shall specify that if *pop* is applied to the empty stack then the empty stack is returned. (So the value returned by *pop* will be $[n_1, n_2, \dots, n_{k-1}]$ for a non-empty stack receiver, and [] for an empty stack receiver.)
4. *peek()* is a method that, for a non-empty stack object, returns its top element. (So if the state of the receiver is $[n_1, n_2, \dots, n_{k-1}, n_k]$ then n_k is returned.) Here we will require that an error message is returned if an attempt is made to apply *peek* to an empty stack.
5. *size()* is a method that returns the number of elements in a stack object. (The size of the empty stack is 0.)

This is slightly different from the primitive type **Stack** described in Unit 3, where a precondition ruled out the possibility of the empty stack being an input to *POP* or *PEEK*.

By specifying methods with precondition **true** we will avoid any issues arising from the use of non-trivial preconditions.

The above method descriptions are independent of the form of data stored in the stack, but for simplicity we will restrict ourselves in what follows to stacks storing integers. Note that the methods all return a value; none changes the state of the receiver.

One of the advantages of having specifications of *push* and *pop* that return a stack is that it permits the stack methods to be composed. That is, if a method returns a value which is a stack, then it can act as the receiver for another method of the class **StackC of Int**. We can continue to compose methods in this way, to form a **chain** such as that below.

new().push(elem1).push(elem2).pop()

We should read such a chain from left to right. First a new (empty) stack is formed. This stack is the receiver for the first *push* with parameter *elem1* (and so returns a stack with state [*elem1*]). This stack becomes the receiver for the second *push* with parameter *elem2* (and so returns a stack with state [*elem1*, *elem2*]). This becomes the receiver for the call of *pop*, and the result is to return a stack with state [*elem1*].

We now take the specification one step further by introducing some formal concepts, such as the method signatures and the sets involved.

Algebraic specification

The formal specification we will give for **StackC of Int** (which we refer to as an **algebraic specification**) will consist of four distinct parts.

- NAME
- SORTS
- SIGNATURES (or syntax)
- AXIOMS (or semantics)

The **sorts** are the sets involved in the signatures of the methods. In the sorts, we include S , which consists of all possible states of an object of type **StackC of Int**.

For example, the method *pop* refers to the state of the receiving stack, and returns a stack value. We give its signature as $pop : S \rightarrow S$. For any method addressed to a receiving stack object, S will be given as the first input set in the signature. If a method has a parameter, as *push* does, then the set from which this is taken will be given after S . So, *push* has input set $S \times Int$, and signature $push : S \times Int \rightarrow S$. (If we were to consider methods that change state as well as those that return a value, we would need to deal with each form of behaviour — as mentioned in Unit 4, p. 13. For simplicity, we only consider here methods that return a value and do not change the state.)

The signatures show how the methods can validly be combined, and may be said to provide the **syntax**. The final part of the specification, the axioms, give the **semantics** — the behaviour of the methods. We shall see shortly how this is done, but for now we just give the first three parts of the specification. As well as S and Int , we also need a sort to carry the error message that may be returned by *peek*. This is the set M below, which consists of a single element, the error string as shown.

NAME	StackC of Int
SORTS	Int S $M = \{\text{"Error : Stack is empty"}\}$
SIGNATURES	$new : \rightarrow S$ $push : S \times Int \rightarrow S$ $pop : S \rightarrow S$ $peek : S \rightarrow Int \amalg M$ $size : S \rightarrow Int$

In general, an algebraic specification will also include preconditions. For simplicity, all the methods in our specification are total functions, with precondition **true**, and we have omitted this aspect of the specification from our example.

Here are some observations concerning the signatures given above.

1. As noted earlier, *new* requires no receiver and no parameters — and so has no input set. This is shown in the signature by the blank space before the arrow. This indicates that the empty stack is created from nothing.
2. *push* has a stack as its receiver and an input parameter from Int ; hence the input set is given as the Cartesian product $S \times Int$. It returns a stack; hence the output set is S .
3. *pop* has a stack as its receiver, and no input parameter, and so has input set S . It returns a stack; hence the output set is S . (If the receiver is the empty stack then the returned value is still a stack — the empty stack.)
4. *peek* has a stack as its receiver and no parameter, and so the input set is S . It returns a value that may be an integer or may come from the set M (the error message). So we give the output set as the disjoint union $Int \amalg M$. This concept was briefly mentioned on p. 22 of Unit 1 and

is used to combine sets containing different types of elements. Here it indicates that the return value is either an integer or the error string.

5. *size* again has a stack as its receiver and no parameter, and so the input set is S . It returns a value from Int .

A chain formed by combining the methods in a way that is consistent with the given signatures is said to be **syntactically correct**. (Remember that a chain is a sequence of method calls, and should be read from left to right.)

EXAMPLE 2.1

- (a) The chain *new().push(11).push(22)* is syntactically correct.

The method *new* requires no input, and returns a stack. The first *push* requires as inputs: a stack (which here is that returned by *new*) and a parameter (which must be a value from Int , and here is 11). This returns a stack, which, together with the integer 22 providing the parameter, are the correct type of inputs for the second *push*. So we see that the chain is syntactically correct (and returns a stack).

- (b) The chain *new().push(33).push()* is *not* syntactically correct.

The second *push* has no integer parameter provided.

- (c) The chain *new().size().push(22)* is not syntactically correct.

new().size() is syntactically correct, and returns a value in Int . This is not the correct type to act as an input to *push(22)*, which requires a stack as receiver, not an integer.

Activity 2.1

Use the signatures to verify that the chain below is syntactically correct, and to determine what type of value it will return.

new().push(17).push(200).size()

Discussion The method *new* requires no input, and returns a stack. The first *push* requires as inputs: a stack (which here is that returned by *new*) and a parameter (which must be a value from Int , and here is 17). This returns a stack, which together with the integer 200 providing the parameter are the correct inputs for the second *push*. This returns a stack, which is the correct type to act as receiver for the call of *size*, and that call will return an integer. So we see that the chain is syntactically correct, and returns an integer. (We expect that the returned value will be 2, but there is as yet nothing in our algebraic specification to tell us that!)

The methods *new* and *push* are called **generators**. We will make an assertion, that:

“all possible stacks in S can be generated using just these two methods”.

Spend a moment or two trying to convince yourself of why this should be true. The completed specification will enable us to justify this statement.

The axioms

The final part of our formal specification will consider how the various methods should relate to one another. This is not something we have considered before in the course. However, we can see that the five methods are not independent of one another.

We will determine how they should relate by reference to the rough specification given above. That tells us informally how we want the methods to behave — and we now wish to express this behaviour formally. We will express the interdependence using the logic notation introduced in Block 3.

We will start by considering how the methods *size*, *peek* and *pop* should interact with the generator method *new* that creates the empty stack.

First we consider *size*. The empty stack should always have size 0 (see rough specification item 5 on page 9). That is, 0 should always be the result of applying the method *size* to the value returned by *new*. This can be expressed formally as follows: the proposition below must always be **true**.

$$0 == \text{new}().\text{size}()$$

Each of our statements about how the methods of **StackC** should interact will be expressed as a proposition that is required always to be **true**. We refer to these **true** propositions as **axioms** of the formal specification.

$$0 == \text{new}().\text{size}()$$

AXIOM 1

In expressing the axioms we can use the language of formal logic introduced in Units 9 and 10. We will also need to be able to compare two values of any particular type. (In Axiom 1, two integers are compared.) We shall write all such comparisons as `==`. In some axioms we shall also need to use operations on integers. We shall assume that the operations on integers are already understood — that is, they are part of the language already available to express formal propositions.

One way of interpreting Axiom 1 is this. If we form a valid chain of methods of the type **StackC of Int**, then we can replace *new().size()* by 0 without changing the value returned by the chain.

Now consider the interaction of *peek* and *new*. Item 4 in the rough specification on page 9 indicates that if we apply the method *peek* to the empty stack we should get an error message, given by the single string in the set *M*. This leads to our second axiom.

$$\text{"Error : stack is empty"} == \text{new}().\text{peek}()$$

AXIOM 2

Activity 2.2

Item 3 in the rough specification on page 9 indicates that if we apply *pop* to an empty stack, then an empty stack is returned. Give a corresponding axiom that shows the relationship between *new* and *pop*.

Discussion The method *new* returns the empty stack, and *pop* applied to the empty stack returns the empty stack. That is, the stack returned by *new().pop()* is the same as that returned by *new()*. So we obtain the following **true** proposition.

$$\text{new}() == \text{new}().\text{pop}()$$

AXIOM 3

(As noted above, in expressing the axioms we will use `==` to express equality of items of any type. Here, we are expressing equality of two objects of type **StackC of Int.**)

The above three axioms show how the methods *size*, *peek* and *pop* interact with the generator *new*. Next we will consider how the methods *size*, *peek* and *pop* interact with the other generator, *push*.

If we push an integer onto a stack the size increases by one (see items 2 and 5 in the rough specification). This suggests the following formalism (where *s* is a stack and *elem* is an integer).

$$s.\text{push}(\text{elem}).\text{size}() == s.\text{size}() + 1$$

If we push *elem* onto the stack *s* and find the size of the resulting stack, then this should be 1 greater than the size of the original stack *s*. This should be the case whatever integer *elem* is and whatever stack *s* is.

The expression above contains free variables (so it is a predicate, not a proposition). To obtain a proposition to act as an axiom, we need to bind these variables with quantifiers.

$$\forall s \text{ in } S [\forall \text{elem in } \text{Int} [s.\text{size}() + 1 == s.\text{push}(\text{elem}).\text{size}()]]$$

AXIOM 4

We now look at the interaction of *push* and *peek*. If we push an integer onto a stack, then the top element of the returned stack is this integer. We can access the top element using *peek*. (See items 2 and 4 in the rough specification.) This suggests the following formalism (where *s* is a stack and *elem* is an integer).

$$\text{elem} == s.\text{push}(\text{elem}).\text{peek}()$$

If we push *elem* onto the stack *s* and peek at the resulting stack, then we should see *elem*. This should be the case whatever integer *elem* is and whatever stack *s* is. Again, to obtain the proposition that we want for our axiom, we need to bind the variables with quantifiers.

$$\forall s \text{ in } S [\forall \text{elem in } \text{Int} [\text{elem} == s.\text{push}(\text{elem}).\text{peek}()]]$$

AXIOM 5

Finally, consider the interaction of *push* and *pop*. If we push an integer onto a stack and then pop the stack, the result should be the same as the stack we started with. (See items 2 and 3 in the rough specification.) We have just added an element and then taken it away.

Activity 2.3

Give an axiom showing the interaction of *push* and *pop*.

Discussion The required axiom is given below.

$$\forall s \text{ in } S [\forall elem \text{ in } Int [s == s.push(elem).pop()]]$$

AXIOM 6

We can now put together a complete formal (or abstract) specification for the type **StackC of Int**.

Abstract specification

NAME **StackC of Int**

SORTS *Int*

S

$M = \{\text{"Error : Stack is empty"}\}$

SIGNATURES *new* : $\rightarrow S$

push : $S \times Int \rightarrow S$

pop : $S \rightarrow S$

peek : $S \rightarrow Int \amalg M$

size : $S \rightarrow Int$

AXIOMS

$0 == new().size()$ (A1)

"Error : stack is empty" == *new().peek()* (A2)

new() == *new().pop()* (A3)

$\forall s \text{ in } S [\forall elem \text{ in } Int [s.size() + 1 == s.push(elem).size()]]$ (A4)

$\forall s \text{ in } S [\forall elem \text{ in } Int [elem == s.push(elem).peek()]]$ (A5)

$\forall s \text{ in } S [\forall elem \text{ in } Int [s == s.push(elem).pop()]]$ (A6)

As noted earlier, an algebraic specification will in general also include preconditions.

To summarise, this formal specification gives the following. The sorts (the sets involved in the signatures); the method signatures (which determine syntax); and the axioms (which determine semantics). To determine appropriate axioms, we referred to the informal specification, in which the semantics were described in natural language. We considered the effect of applying each of the methods *size*, *peek* and *pop* to each of the generators *new* and *push*, and looked for a simplification of the result.

Completeness

At the start of the section we said that the formal specification should give a description that is as complete as possible. How can we be sure that we have all the axioms we need for a complete description?

The set of axioms is said to be **complete** if:

- they define the outcome of all permissible applications of the methods of the abstract data type **StackC of Int**;
- composition of the methods (in chains) constructs all possible states of an object of type **StackC of Int**.

The six axioms given for **StackC of Int** do satisfy the first of these criteria.

For instance, we might think it necessary to include an axiom that specifies the interaction of the methods *pop* and *size*. Informally, we can see that when a non-empty stack is popped the size of the returned stack decreases by 1 — but there is no axiom corresponding directly to this. However, this

follows from the given axioms. Suppose that s is any stack, whose size is k (so $k \geq 0$). Then by (A4), $s.push(elem)$ gives a stack of size $k + 1$. By (A6), $s.push(elem).pop()$ is s , and so has size k . Now any non-empty stack will be of the form $s.push(elem)$ for some stack s and integer $elem$ (see below). Accepting that, we see that applying pop to any non-empty stack reduces the size of the stack by 1; that is, this result is a consequence of the given axioms (A1)–(A6) and does not need to be given as an additional axiom.

A mathematician would call such a consequence of the axioms a **theorem**.

We saw earlier that the signatures enable us to recognise when a chain of methods is syntactically correct. Given a syntactically correct chain, the axioms may enable us to rewrite it. For example, suppose that $elem1$ and $elem2$ are integers. Then we can use the signatures to verify that the chain $new().push(elem1).push(elem2).size()$

is syntactically correct (see Activity 2.1), and to see that the chain should return an integer. We can use the axioms to rewrite this chain as follows.

$$\begin{aligned}
 new().push(elem1).push(elem2).size() &= new().push(elem1).size() + 1 && \text{(by A4)} \\
 &= (new().size() + 1) + 1 && \text{(by A4 again)} \\
 &= (0 + 1) + 1 && \text{(by A1)} \\
 &= 2.
 \end{aligned}$$

This is certainly the expected result! Working informally, we expect that $new()$ will return the empty stack $[]$, then $new().push(elem1)$ will return the stack $[elem1]$, and finally $new().push(elem1).push(elem2)$ will return the stack $[elem1, elem2]$, which contains two elements, and so has size 2. The algebra above shows that this expected result can be obtained by an argument based solely on the axioms. In the formal argument, the axioms are used to replace (or rewrite) parts of the chain. Indeed, such axioms are sometimes called **rewrite rules**.

At this stage, we are using the notation $[elem1, elem2]$ in a way separate from the algebraic specification. However, we can tie it more formally to that specification quite easily. We can say that we will use the following notation for the stack values returned by certain chains.

stack value	chain
$[]$	$new()$
$[elem1]$	$new().push(elem1)$
$[elem1, elem2]$	$new().push(elem1).push(elem2)$
$\vdots$	and so on

This convention provides a notation for the stack value returned by any chain that starts with new and follows that with a series of calls of $push$. According to our informal view of the class, these values constitute “all possible values” for a stack. This suggests, informally at least, that we can regard our axioms as satisfying the second criterion for completeness given above. It also clarifies how we may interpret the first criterion for completeness. Do the axioms determine the value returned by any valid chain, where the value returned may either be an integer, or the error message value in the set M , or a stack value as represented by the above notation?

We can see that this must be the case. If a valid chain includes either $size$ or $peek$ then the returned value must lie in either M or Int . (We can see this by examining the signatures.) If a valid chain contains only new and $push$, then it must be of the form just discussed. This leaves only valid chains formed using new , $push$ and pop . For such a chain, axioms (A3) and (A6)

permit us to remove all the occurrences of *pop*, and leave a valid chain just involving *new* and *push*. Consequently, we can deduce the following result.

Canonical form result

Any valid chain that returns a stack value can be rewritten by use of the axioms in the form below, in which the chain involves n calls of *push* (where $n \geq 0$), and *elem1*, *elem2*, ..., *elemn* are integers.

new().push(elem1).push(elem2).push(elem3)....push(elemn)

We call *new* and *push* generators for **StackC of Int** because all possible stack values can be produced using just these two methods.

The expression

new().push(elem1).push(elem2).push(elem3)....push(elemn)

given in the above result is called the **canonical form** of a chain. For convenience, we shall write such a canonical form as

[*elem1*, *elem2*, ..., *elemn*]

The term **normal form** is sometimes used instead of canonical form.

The proof of the canonical form result that we gave above may seem rather informal. It can be made formal by use of mathematical induction on the number of method calls in the chain.

Activity 2.4

Use the axioms to simplify the following chain, and write down the representation of the stack it returns.

new().push(11).push(22).pop().push(33)

Discussion We can use (A6) to remove *push(22).pop()*.

new().push(11).push(22).pop().push(33) = new().push(11).push(33) (by A6)

The resulting stack value can be written as [11, 33].

Activity 2.5

Write down the chain in canonical form that generates the stack [2, -3, 4].

Discussion We push the elements one-by-one onto the empty stack.

new().push(2).push(-3).push(4)

Our algebraic specification of **StackC of Int** is now complete. The algebraic specification is said to produce an **algebra** on the sorts. We are able to use the axioms and signatures to manipulate chains of method calls, determining which are valid, and replacing certain combinations (and so simplifying chains if we wish to).

Such a formal specification provides sufficient information to enable us to write code using it. (Of course, for such code to run, say in the WorkPad, we would need to provide an implementation of the class. An implementation of **StackC of Int** is provided in the practical computing material.)

Activity 2.6

Give two implementations of the function specified below (one iterative and one recursive).

function *STACKSUM*(*s* in **StackC** of **Int**) **return** in **Int**

pre true.

post The returned value is the sum of the integers in the stack *s*. (The returned value is 0 if *s* is empty.)

Discussion We give the iterative implementation first.

Remember that the methods of **StackC**, *pop* in particular, are value returners, not state changers.

```
function STACKSUM(s)
{
    //Iterative implementation
    var t in StackC of Int
    var sum in Int
    sum <-- 0
    t <-- s.clone()
    while (NOT(t.size() == 0))
    {
        sum <-- sum + t.peek()
        t <-- t.pop()
    }
    return sum
}
```

```
function STACKSUM(s)
{
    //Recursive implementation
    if (s.size() == 0) then
    {
        return 0
    }
    else
    {
        return s.peek() + STACKSUM(s.pop())
    }
}
```

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ In the context of an algebraic specification, understand the terminology sorts, signatures (syntax) and axioms (semantics).
- ☐ Understand what the generator method *new* of **StackC** of **Int** does, and interpret its signature.
- ☐ For a chain of calls of methods of **StackC** of **Int**: test whether the chain has valid syntax; simplify a valid chain using the axioms; determine the value returned by a valid chain (if necessary, given using the notation $[elem1, elem2, \dots, elemn]$ for a stack value).
- ☐ Give an implementation using the type **StackC** of **Int**.

Exercises on Section 2

Exercise 2.1

For each of the following chains, check whether the syntax is correct and, if it is, use the axioms to determine the value returned.

- (a) `new().push(100).push(200).peek()`
- (b) `new().pop().peek()`
- (c) `new().push(100).peek().size()`
- (d) `new().push(14).pop().pop().push(99)`

Exercise 2.2

Give two implementations of the function *NSTACK* whose specification is given below, the first using a **for** loop, and the second recursive.

function *NSTACK*(*n* in Int) **return** in StackC of Int

pre $n \geq 0$.

post The returned value is the stack $[1, 2, 3, \dots, n]$, containing n integers in the order shown.

3 Fuzzy logic

When studying Units 9 and 10 you may have felt constrained by the requirement that a proposition must be either **true** or **false**. Partly, of course, the truth value may depend on the circumstances. If these circumstances can be expressed using parameters, then the ideas of predicate logic will help. However, consider the following three assertions.

The character at position 8 in the string *myString* is in upper case.

I have given the mouse the second click of a double click.

“The quick brown fox jumped over the lazy dog” is a long sentence.

In the first example, if *myString* is less than 8 characters long then the truth value of the proposition is problematic. One way of dealing with situations like this is to introduce a third possible truth value, namely **undefined**, into your logic. Decisions have then to be made as to how to work with this new value when using the logical operators $\wedge$, $\vee$ and $\neg$. The resulting system is known as a **three-valued logic**. The formal specification language VDM makes use of a logic of this form (see **Jones**, p. 75.) Indeed, this idea can be extended to allow for more than three possible truth values for propositions. Logics using these ideas are known as “multi-valued logics”.

In the second example, the assertion is time dependent. The second click of the mouse has to happen sufficiently briefly after the first for it to be counted as the second click of a double click. That is, there is a brief interval after the first click during which the statement “Clicking the mouse means that a double click has occurred” has value **true**. A logic that deals with ideas like this, whose truth value changes with time, is known as **temporal logic**. In such a logic, truth values like **sometime before**, **sometime after**, **always**,

never and **until** may be introduced. This allows for reasoning about time-dependent systems and has a particular role where concurrent systems are involved.

In the final example, the problem lies with our understanding of the word “long”. The traditional approach to evaluating the truth of propositions containing such words is to make a decision about what is meant by “long”, that is essentially arbitrary. This might be, for example, that any sentence containing more than 20 words is long. Such decisions are inflexible and allow for no gradations of meaning. In the 1960s a new logic was introduced that would allow us to capture these gradations. This logic is known as **fuzzy logic**. This section gives a brief introduction to the ideas involved.

In 1965, Lotfi Zadeh published a paper called “Fuzzy sets”. This was followed, in 1973, by a paper called “Fuzzy languages and their relation to human and machine intelligence”. These papers introduced the underlying ideas of what is known as **fuzzy logic**. When we looked at the traffic problem in Unit 9 we had conditions such as “if the queue of cars is long”. In Unit 9 this had to be translated into “if the queue of cars has more than *number* cars” where *number* took on a specific value. This does not allow for any vagueness in what we mean by “a long queue”. Although we might be sure that a queue of 2 cars is not long and that a queue of 50 cars is long, we might be less sure for numbers in between. The idea of fuzzy logic is to develop techniques that allow for less sharp cut-off points. Thus fuzzy logic is about translating vagueness. It can also be made to be dynamic and so respond to changing situations. For example, a tumble dryer with fuzzy logic engineering (together with appropriate sensors) will tumble clothes until they are dry and then stop. The display will show an estimate of the time needed. This estimate is constantly updated while the clothes are drying. The updated values are calculated using the current state of the clothes. The vagueness here is in the interpretation of how dry they are and how much longer they will need. The calculation will depend on so called **fuzzy rules**. These might be as follows.

If the clothes are very wet they will need a long time to dry.

If the clothes are wet they will need quite a long time to dry.

Fuzzy logic will allow interpretations of “very wet” and of “wet” that can overlap. This allows greater flexibility.

When one uses such a tumble dryer, one does not have to think about how long to give the clothes nor to keep checking their state. The dryer stops when they are dry and constantly gives an estimate of how soon this will be.

If we used standard rules (as in Unit 9) the dynamic response would be liable to sudden changes. The advantage of the fuzzy rules is that they allow a smoother response.

In this section we will give a brief introduction to the techniques used and carry out a small case study. The purpose here is to give you a rough idea of what fuzzy logic involves, but no more.

Fuzzy rules and states

In traditional logic conditions are either **true** or **false**. In fuzzy logic boundaries are more blurred. This is best illustrated by an example. Suppose that we wish to control a heater. We would have some fuzzy rules, such as the following.

- If the room temperature is cold then we need a high power output.
- If the room temperature is cool then we need a moderate power output.
- If the room temperature is warm then we need a low power output.

We now need to interpret the words “cold”, “cool”, “warm”. One approach to this situation is to use overlapping states. We name the states appropriately as *cold*, *cool*, and *warm*. We then determine a range of values for each of these states, say less than 14 for *cold*, 12–18 for *cool* and more than 17 for *warm*. The states are overlapping: for example, if the temperature is 13 then it comes into the ranges for both *cold* and *cool*. We then determine states for the power output. These can also be overlapping. Both sets of states are then used to calculate power output as the system is running. This is done using the fuzzy rules.

These numbers are temperatures in degrees C (for Celsius).

Of course, how well the system runs depends on the number of states used and on the ranges used for those states. Decisions about this are beyond the scope of this discussion. The remainder of this section is a description of how to process a particular example using fuzzy logic techniques. The values given in the examples chosen are for illustration only.

A case study in fuzzy logic

We wish to control a set of traffic lights according to the build-up of traffic. We will assume that we have a sensor that can detect the length of a traffic queue. The length will take values from 0 to 100 units. The controller is to set the period of time for which the traffic lights will stay green in units from 0 to 100. (We will suppose that 0 corresponds to a preset minimum length of time and that 100 corresponds to a preset maximum length of time.) We will show how we can use the ideas of fuzzy logic to obtain an output giving the length of time the light stays green from an input giving the length of traffic queue.

Two different sets of units are involved here — one for queue lengths and one for the period of time.

As a first attempt, we will start with three fuzzy rules, given below.

- If the queue is short the lights are on for a short time.
- If the queue is medium the lights are on for a medium time.
- If the queue is long the lights are on for a long time.

This means that we need three states for the traffic queue, and we need to relate these to our scale of 0–100 units.

- *short*, corresponding to 0–40
- *medium*, corresponding to 25–75
- *long*, corresponding to 60–100

We also need three states for the lights.

- *short time*, corresponding to 0–40
- *medium time*, corresponding to 25–75
- *long time*, corresponding to 50–100

Suppose that the traffic queue has length 62. How long should the lights be green?

Notice that although our rules are fuzzy our actual input (and the corresponding output) are **crisp** — a word used in this context to indicate that an exact value is being used.

The first step is to determine where the input value lies in terms of the input states. We do this by using triangles to represent the states. Each state is represented by a triangle. This has height 1 unit, and a base corresponding to the range of values included in the state. The highest point of the triangle is usually placed above the centre of the base. However, the triangle for the lowest state often has its highest point above the left end of the base, and that for the highest state over the right end of the base, and we have used this approach.

Thus the triangles for the queue can be set up as shown in Figure 3.1.

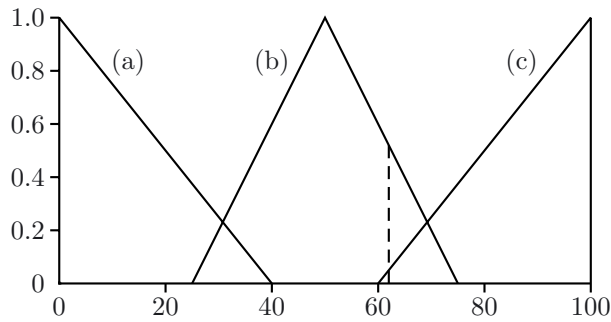


Figure 3.1 Triangles for the queue states: (a) *short*; (b) *medium*; (c) *long*.

Now the input value 62 lies in the range for two of the three possible states.

62 is **not** between 0 and 40;
 62 **is** between 25 and 75;
 62 **is** between 60 and 100.

So 62 falls in the triangles corresponding to *medium* and *long* queue states. However, 62 is “only just” in the triangle representing the *long* state. The next step is to determine “how much” 62 belongs to each of the triangles. We do this by considering the height of each triangle above the point 62. For the *medium* triangle this height (shown by the dotted line in the picture) is 0.52, and for the *long* triangle the height is 0.05.

You are not expected to do such calculations yourself.

We now wish to determine the output value corresponding to an input of 62. To do this, first draw a triangle for each possible output state. This approach is similar to that for the input states, and the triangles are shown in Figure 3.2.

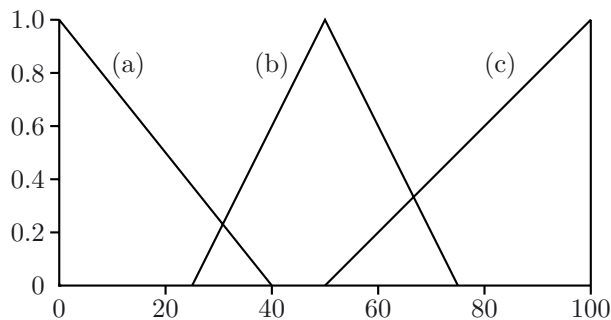


Figure 3.2 Triangles for the output states: (a) *short time*; (b) *medium time*; (c) *long time*.

Now we saw that the input value 62 is not *small*; is *medium* with a weighting of 0.52; and is *large* with a weighting of 0.05. The three triangles in Figure 3.2 show the output corresponding to each of these queue states. We now discard the *short time* triangle, and scale the other two in accordance with these weightings — see Figure 3.3(a).

The output corresponding to an input 62 is then the “average” of these output triangles shown in Figure 3.3(a). This average is determined by first “adding” the triangles — the result is shown in Figure 3.3(b). We then determine the point on the horizontal axis at which a vertical line (shown dashed in Figure 3.3(b)) divides the region into two equal areas. This process is known as **defuzzifying**.

This is just one of various techniques that may be used for determining the output.

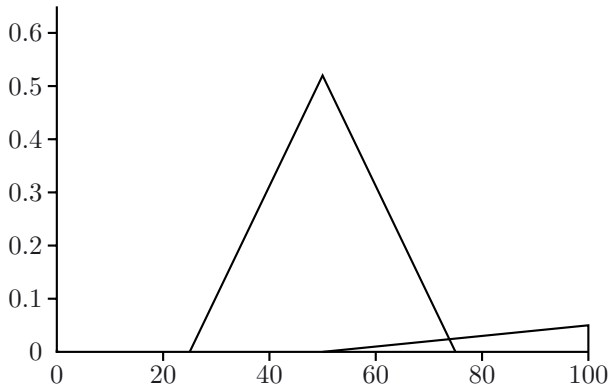


Figure 3.3 (a) The output triangles scaled for a queue length of 62.

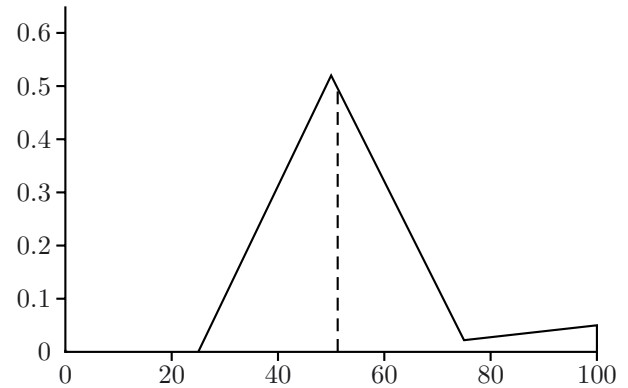


Figure 3.3 (b) The result of “adding” the two triangles in (a).

For an input of 62, this process gives the output value 51.23. Notice that for each input this approach calculates a precise output.

For any queue length between 0 and 100, we can use the same approach to calculate the time that the lights should stay green. Figure 3.4 shows this calculated length of time for different lengths of queue.

Increasing the number of rules

If we increase the number of descriptions of the queue length to 5 and the number of descriptions of traffic light length to 5 then we can obtain more delicate control. For example, suppose that we have the following five fuzzy rules.

- If the queue is short the lights are on for a short time.
- If the queue is fairly short the lights are on for a fairly short time.
- If the queue is medium the lights are on for a medium time.
- If the queue is fairly long the lights are on for a fairly long time.
- If the queue is long the lights are on for a long time.

We might then have the following five states for both the traffic queue and the traffic lights.

- *short*, corresponding to 0–50
- *fairly short*, corresponding to 10–65
- *medium*, corresponding to 45–75
- *fairly long*, corresponding to 60–90
- *long*, corresponding to 80–100

The lights are described being on for a

- *short time*, corresponding to 0–30
- *fairly short time*, corresponding to 10–50
- *medium time*, corresponding to 40–60
- *fairly long time*, corresponding to 60–90
- *long time*, corresponding to 80–100

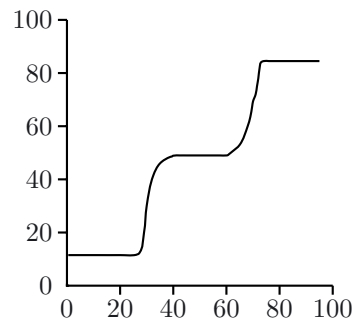


Figure 3.4 Output values with three rules. (The units for each scale have not been specified.)

Again, for any queue length between 0 and 100, we can use the same approach to calculate the time that the lights should stay green. The output values we get in this case are shown in Figure 3.5. For example, an input of 48 gives rise to an output of 31.16.

Clearly the efficacy of this approach depends on decisions about the number of rules and the representation of the terms involved as intervals. Such decisions are beyond the scope of this course.

The system we have described is essentially one-dimensional. Many systems where fuzzy logic is used will be multi-dimensional. The traffic light example would become multi-dimensional if the rules took account not only of the traffic at the given lights but also of the traffic at opposing lights and, perhaps, of any build-up elsewhere in the local road system. Implementing these rules requires generalising the approach with the triangles to three or more dimensions. A good starting point for further reading is **BISC**.

One final remark: fuzzy logic is a practical tool, and is being used in applications from car transmission systems to stock trading!

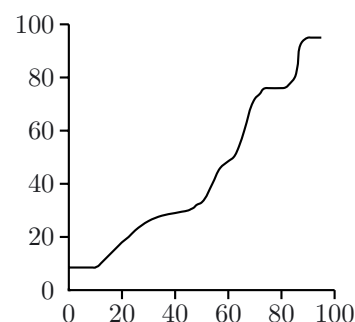


Figure 3.5 Output values with five rules.

4 Beyond big Oh

In Unit 15 we investigated the efficiency of algorithms, including one for the famous Travelling Salesman Problem (TSP). We saw that, even for small problem sizes, the TSP became intractable with our naive algorithm. We stated that there were better algorithms for versions of the problem, but that these could not be guaranteed to find the optimal route. In fact, we know of no algorithm that is guaranteed to find the shortest route and that will run in a reasonable amount of time for large problem sizes.

We are now coming to the point of recognising which problems it is feasible to solve using current digital computing technology. The truth is, we have no very well defined measure of feasibility, but there is a generally accepted “line in the sand”. By convention, it is reckoned that if an algorithm requires polynomial time, it is feasible to run it and obtain its result. If it is worse than polynomial (such as factorial or exponential), as we saw for our algorithm for the TSP, it quickly becomes impractical, even for quite small instances of the problem. In fact, even time complexities worse than cubic (such as n^4) may not be very useful in practice. We denote by **P** the class of problems for which there is a known algorithm that can be run in polynomial time. Problems in **P** are called **feasible** or **tractable**.

You may ask if circumstances might not get better in the future. Surely our computers will continue to get more powerful, as has been the case since their invention? Would that not solve the problem of algorithms with exponential time complexity?

If future computers are built on the same principles as those of today, then they will depend on an ability to switch states rapidly. The purpose of a bit is to store a 0 or a 1, and switch between these. It is the purpose of a processor to process groups of bits. How fast can we make such switches operate? At 10^{15} switches per second, we will have approached the frequency of visible light. At this point, physicists tell us, the energy needed for the switching would break the chemical bonds holding materials together!

We have already achieved 10^9 switches per second (one Gigahertz), so we can expect at best a million times faster processing power. (That is the ratio of the “best possible” to the current speed: $10^{15}/10^9 = 10^6$.)

There is a rule of thumb, known as Moore’s Law (which is not really a law) that every 18 months chip designers are able to fit twice as many transistors on a chip. With transistors being the key component on the chip, doubling chip density is reckoned to double the computing power.

Gordon Moore was an engineer with Intel in 1965.

In fact, it turned out that from 1971 to 1997 doubling in processing power was occurring closer to every 25 months (see **Transistors**, which gives a rate of increase by a factor of 1.38 per year). As an approximation, we can say the doubling period is two years.

How many doublings in processing power can we have, and how long will it take until we achieve it? It appears that we can continue the increase in transistor density up to a million times, before our technology cannot cope with the speed of switching. How many doublings achieve such an increase? If there have been x doublings, then when $2^x = 10^6$ we will have achieved our maximum million-fold increase in processing power. Solving this, we see that $x = \log_2(10^6)$, which is about 20. If each doubling period is about two years, then in roughly forty years we might hit the ceiling of processing power. (That assumes that we continue to use our current computing technology, and that current trends continue.)

How much difference would such new computing power make? Well, if your original hardware could run your program in ten billion millennia, speeding it up by a million times would mean that your new hardware could do the same job in ten thousand millennia. We saw this kind of running time with the TSP in Unit 15. Faster computers are not going to solve this problem!

What about **parallel computing**, where multiple processors are put to work together, or **distributed systems**, where the processing power exists in various physical locations? In general, these techniques may speed processing up by orders of magnitude (e.g. they may make processing thousands of times faster), but this is still not significant within a time scale of ten billion millennia. The time scale to do the computation would remain impractical. Only certain “embarrassingly parallel” problems are especially amenable to faster processing by these techniques.

Two formulations of speedup achievable by parallelism are Amdahl’s Law and Gustafson’s Law. See **Amdahl**.

It is not clear whether quantum computers, if they can be built on a practical scale, will solve this problem.

NP and NP Complete

Imagine a computer that can “magically” choose the correct path to follow when faced with many alternatives. (Or you may think of it as able to follow all alternative paths simultaneously, so that it has magically found which of the alternative paths was worth following.) Such machines are said to be **non-deterministic**. We have no way at present of building a machine that works this way! (**Quantum computing** provides a possible area of future development.)

See **Quantum**.

NP stands for a class of problems called **non-deterministic polynomial**. If we could build a non-deterministic machine, it could solve a problem in **NP** in polynomial time. All the problems in **P** are also in **NP** (because the non-deterministic machine can behave like a deterministic one).

There is another class of problems, **NPC**, a subset of **NP**, called **NP Complete**. These problems are equivalent to one another in the following sense: if one of them can be solved in polynomial time, so can all the others. This arises from the fact that it is possible to reformulate any one problem in this set in terms of any other, in polynomial time. The combined work of polynomial transformation and a polynomial solution would itself

be polynomial. Problems in *NPC* are said to be **computationally hard**. In fact, the Travelling Salesman Problem is in this class.

As we have said, if one of the problems in *NPC* is found to be solvable in polynomial time, so are all the others. This would mean that *P* and *NPC* are not different. However, none of the problems in *NPC* has a known polynomial solution. At the moment it is generally believed that *P* and *NPC* are different, but, so far, we have no proof of this.

Canadian computer scientist Stephen Cook formalised the idea of NP completeness in 1971.

Cook's theorem states that the Boolean Satisfiability problem is NP Complete.

5 References and Further Reading

Amdahl, G.M. "Validity of single-processor approach to achieving large-scale computing capability", in *AFIPS Conference Proceedings*, vol. 30, AFIPS Press, Reston, VA, 1967, pp. 483–485.

Cook, S. "The complexity of theorem proving procedures", in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, ACM Press, New York, 1971, pp. 151–158.

Dijkstra, E. W. "Guarded commands, nondeterminacy and formal derivation of programs", in *Communications of the ACM* vol. 18(8), 1975, pp. 453–457.

Floyd, R. W. "Assigning meanings to programs", in *Proceedings of Symposia in Applied Mathematics*, vol. 19, American Mathematical Society, Providence, RI, 1967, pp. 19–32.

(**Gerhart**) Gerhart, S., Craigen, D. and Ralston, T. "Observations on industrial practice using formal methods", in *Proceedings of the 15th International Conference on Software Engineering*, IEEE Computer Society Press, Los Alamitos, CA, 1993, pp. 24–33.

Gustafson, J.L. "Re-evaluating Amdahl's law", in *Communications of the ACM*, vol. 31(5), 1988, pp. 532–533.

(**Hoare1**) Hoare, C. A. R. "Communicating sequential processes", in *Communications of the ACM*, vol. 21(8), 1978, pp. 666–667.

(**Hoare2**) Hoare, C. A. R. "An axiomatic basis for computer programming", in *Communications of the ACM*, vol. 12(10), 1969, pp. 576–580.

(**Jacky**) Jacky, J. *The Way of Z*, Cambridge University Press, 1997. ISBN 0-521-55976-6.

(**Jones**) Jones, C. B. *Systematic Software Development using VDM*, Prentice-Hall, London, 1986. ISBN 0-13-880733-7.

Kosko, B. *Fuzzy Thinking*, Flamingo, London, 1994. ISBN 0-006547133.

(**Meyer**) Meyer, B. *Object-Oriented Software Construction*, Prentice Hall, London, 1997. ISBN 0-13-629031-0.

(**Milner**) Milner, R. *A Calculus of Communicating Systems*, Springer, Berlin, 1980. ISBN 0-387102353.

Rawlins, G. *Compared to What?*, Computer Science Press, New York, 1992. ISBN 0-716-8243-X.

Riefel, E. and Polak, W. “An introduction to quantum computing for non-physicists”, in *ACM Computing Surveys*, vol. 32(3), 2000, pp. 300–335.

Ross, T. *Fuzzy Logic with Engineering Applications*, McGraw Hill, New York, 1995. ISBN 0-07-053917-0.

Web sources

- (**ADT**) *Abstract data type*, Wikipedia (12 July 2004), **at:** http://en.wikipedia.org/wiki/Abstract_data_type Accessed on 29 July 2008.
- (**Amdahl**) *Amdahl’s Law*, Wikipedia (30 June 2004), **at:** http://en.wikipedia.org/wiki/Amdahl%27s_Law Accessed on 29 July 2008.
- (**BISC**) *The Berkeley Initiative in Soft Computing*, Dr. Masoud Nikravesh, 2002, **at:** <http://www-bisc.cs.berkeley.edu/BISCPprogram/default.htm> Accessed on 29 July 2008.
- (**BPEL4WS**) *Business Process Execution Language for Web Services Version 1.1* (05 May 2003), Tony Andrews *et al*, **at:** <http://www.ibm.com/developerworks/library/specification/ws-bpel/> Accessed on 29 July 2008.
- (**Church_Turing**) *Church–Turing thesis*, Wikipedia (12 July 2004), **at:** http://en.wikipedia.org/wiki/Church_Turing_Thesis Accessed on 29 July 2008.
- (**Nicely**) *Bug in the Pentium FPU*, Thomas Nicely (Lynchburg College, VA, USA) (30 October 1994), **at:** <http://www.trnicely.net/pentbug/bugmail1.html> Accessed on 29 July 2008.
- (**Quantum**) *Certainty from Uncertainty*, Charles H. Bennet (25 August 2003), **at:** <http://www.fortunecity.com/emachines/e11/86/qcomp2.html> Accessed on 29 July 2008.
- (**Transistors**) *Transistors per integrated circuit trends*, **at:** <http://www.icknowledge.com/trends/14074b.pdf> Accessed on 29 July 2008.

Solutions to the exercises

Solution 2.1

(a) $new().push(100).push(200)$ is certainly syntactically correct, and will return a stack. We can then add a call of $peek$ to this, and the value then returned will be in Int . We have:

$$new().push(100).push(200).peek() = 200 \quad (\text{by A5})$$

(b)

$$\begin{aligned} new().pop().peek() &= new().peek() && (\text{by A3}) \\ &= \text{"Error: stack is empty"} && (\text{by A2}) \end{aligned}$$

This evaluation also establishes that the chain is syntactically correct.

(c) The chain $new().push(100)$ is syntactically correct, and returns a value in S . So the chain $new().push(100).peek()$ is syntactically correct, and returns a value in Int . But the input set of $size$ is S (not Int), so the given chain is not syntactically correct.

(d)

$$\begin{aligned} new().push(14).pop().pop().push(99) &= new().pop().push(99) && (\text{by A6}) \\ &= new().push(99) && (\text{by A3}) \end{aligned}$$

We see that this chain is valid, and will return the stack $[99]$.

Solution 2.2

Two implementations are given below.

```
function NSTACK(n)
{
    // Iterative implementation
    var s in StackC of Int
    var i in Int
    for (i <-- 1 to n)
    {
        s <-- s.push(i)
    }
    return s
}
```

```
function NSTACK(n)
{
    // Recursive implementation
    var s in StackC of Int
    if (n == 0) then
    {
        return s
    }
    else
    {
        return NSTACK(n-1).push(n)
    }
}
```

Index

- NPC*, 24
- NP*, 24
- P*, 23

- algebraic specification, 10
 - StackC of Int**, 14
 - axioms, 12
 - completeness, 14
 - generators, 11
 - semantics, 10
 - syntax, 10

- chain (of methods), 9
 - canonical form, 16
 - normal form, 16
 - syntactically correct, 11
- constructor (method), 9

- feasible (problem), 23
- fuzzy logic, 19

- fuzzy rules, 19

- Hoare triple, 6

- non-deterministic machine, 24
- NP Complete, 24

- program derivation, 7

- rewrite rule, 15

- specification
 - algebraic, 10
 - axiomatic, 6
 - explicit, 6

- temporal logic, 18
- three-valued logic, 18
- tractable (problem), 23

- verification, 6